

Diseño e Implementación de un Lenguaje

Fecha: 3 de septiembre de 2026

Nombre del lenguaje: B#

1. Equipo

Nombre	Apellido	Legajo	E-mail
Milagros Maria	Pipet	65104	mpipet@itba.edu.ar
Martina	Malleville	63335	mmalleville@itba.edu.ar
Carolina Luciana	Laurenza	64647	claurenza@itba.edu.ar
Victoria Helena	Park	64498	vpark@itba.edu.ar

2. Repositorio

La solución será versionada en:

<https://github.com/vpark2003/TP-TLA.git>

Nombre del lenguaje: B#

3. Dominio

3.1. Descripción

El dominio seleccionado es la **descripción y estructuración armónica de canciones**, particularmente mediante la representación de canciones en formato de **lead sheet**.

Un lead sheet es una representación simplificada de una composición musical que permite expresar principalmente su estructura, métrica y armonía. En B#, una canción será modelada a partir de:

- Nombre de la canción.
- Tempo.
- Compás.
- Tonalidad.
- Secciones musicales.
- Compases.
- Acordes.

La salida principal del lenguaje será un archivo **MIDI**, generado a partir de la información armónica descrita por el usuario.

El objetivo de B# es permitir que una persona pueda describir la estructura armónica de una canción utilizando una sintaxis cercana a la utilizada por músicos y compositores, sin tener que ocuparse de los detalles técnicos necesarios para representar la música en un formato computacional.

Una canción estará formada por diferentes secciones, como:

- `intro`
- `verse`
- `chorus`
- `bridge`
- `outro`

Cada sección podrá contener una secuencia de compases. Los compases estarán delimitados mediante el símbolo `|` y contendrán uno o más acordes.

Por ejemplo:

```
section verse x2 {  
  | C | Am | F | G |  
}
```

La sintaxis permite representar directamente una progresión armónica de cuatro compases, haciendo que el programa sea visualmente similar a una representación musical simplificada.

El lenguaje también permitirá especificar la cantidad de repeticiones de una sección:

```
section chorus x2 {  
  | F | G | Em | Am |  
}
```

De esta forma, una misma estructura musical puede reutilizarse sin necesidad de escribirla múltiples veces.

Una característica central de B# será la posibilidad de especificar la duración de cada acorde dentro de un compás. Esto permitirá que un mismo compás pueda contener más de un acorde:

```
| C(2) G(2) |
```

En un compás de 4/4, el ejemplo anterior indica que el acorde `C` ocupa 2 tiempos y `G` ocupa los 2 tiempos restantes.

El compilador podrá comprobar que la suma de las duraciones de los acordes de cada compás sea compatible con el compás declarado.

Por lo tanto, B# será un **Domain Specific Language (DSL)** específicamente orientado a la descripción de estructuras armónicas musicales, utilizando conceptos propios del dominio como canciones, secciones, compases, acordes, tonalidades y tiempos.

3.2. Motivación

La representación computacional de una canción puede requerir trabajar con información técnica como números MIDI, posiciones temporales, duraciones expresadas en unidades internas y estructuras de datos.

Estos detalles no necesariamente forman parte del modelo mental de una persona que está describiendo la armonía de una canción.

B# busca abstraer estos aspectos y permitir una descripción más cercana al lenguaje utilizado en una práctica musical.

Por ejemplo, en lugar de especificar individualmente los valores MIDI correspondientes a cada nota de un acorde, el usuario puede escribir:

```
| C | Am | F | G |
```

Esta representación permite identificar inmediatamente una progresión armónica.

Además, B# permite expresar la estructura temporal de los compases:

```
time 4/4  
  
| C(2) G(2) |  
| Am(4)      |
```

La primera línea representa un compás en el que **C** ocupa dos tiempos y **G** ocupa los dos restantes, mientras que la segunda representa un compás completo ocupado por **Am**.

La motivación principal del lenguaje es, entonces, **reducir la distancia entre la forma en que un músico piensa una canción y la forma en que esta debe ser expresada computacionalmente**.

3.3. Especificidad del dominio

El dominio es específico porque todas las principales construcciones del lenguaje representan conceptos propios de la música y, particularmente, de la representación armónica de canciones.

Las principales entidades del dominio presentan las siguientes relaciones:

- Una canción está formada por secciones.
- Una sección está formada por compases.
- Un compás contiene uno o más acordes.
- Un acorde posee una raíz y una cualidad.
- Un acorde puede poseer extensiones.
- Un acorde puede especificar una nota diferente en el bajo.
- El compás determina la cantidad de tiempos disponibles en cada compás.
- El tempo determina la velocidad de reproducción.
- La tonalidad permite describir el contexto tonal de la composición.
- Una sección puede repetirse una determinada cantidad de veces.

Por lo tanto, el lenguaje puede representar directamente estos conceptos como abstracciones del modelo computacional.

Una de las características distintivas del dominio será la relación entre **acordes, compases y duración temporal**.

Por ejemplo:

```
time 4/4  
| C(2) G(2) |
```

es válido porque:

```
2 + 2 = 4
```

mientras que:

```
| C(3) G(2) |
```

deberá ser rechazado porque:

```
3 + 2 = 5
```

y el compás dispone únicamente de cuatro tiempos.

Esta validación constituye una de las principales reglas semánticas específicas de B#.

4. Construcciones, prestaciones y funcionalidades

B# deberá permitir las siguientes construcciones.

4.1. Definición de una canción

Se podrá crear una canción indicando su nombre:

```
song "Midnight" {  
  ...  
}
```

El bloque de la canción contendrá la configuración general de la composición y las diferentes secciones que la forman.

4.2. Configuración del tempo

Se podrá establecer el tempo de una canción mediante BPM:

```
tempo 120
```

El valor deberá ser un número entero positivo.

El tempo será utilizado posteriormente durante la generación del archivo MIDI.

4.3. Definición del compás

Se podrá indicar el compás de la composición mediante un numerador y un denominador:

```
time 4/4
```

El numerador representa la cantidad de tiempos disponibles en cada compás.

El denominador representa la figura musical que recibe un tiempo.

El denominador deberá corresponder a una potencia de dos, como:

```
2  
4  
8  
16
```

Por ejemplo:

```
time 4/4
```

representa un compás de cuatro tiempos, donde cada tiempo corresponde a una negra.

4.4. Definición de la tonalidad

Se podrá especificar la tonalidad de la composición:

```
key C major
```

También podrán utilizarse tonalidades menores:

```
key A minor
```

La tonalidad permitirá contextualizar la armonía de la canción y podrá ser utilizada durante el análisis semántico.

Por ejemplo, el compilador podrá identificar si los acordes utilizados pertenecen a la tonalidad declarada.

La comprobación de acordes no diatónicos podrá tratarse como una advertencia o como una validación estricta, según la implementación final.

4.5. Secciones musicales

La estructura de una canción podrá dividirse en secciones identificables.

Cada sección tendrá un nombre y contendrá sus compases:

```
section verse {
    | C | Am | F | G |
}
```

Se contemplarán como conceptos de sección, entre otros:

- `intro`
- `verse`
- `chorus`
- `bridge`
- `outro`

El identificador de la sección permitirá reconocer la función que cumple dentro de la estructura de la canción.

4.6. Repetición de secciones

Una sección podrá indicar directamente la cantidad de veces que debe repetirse:

```
section chorus x2 {
    | F | G | Em | Am |
}
```

El `x2` indica que la sección deberá ejecutarse dos veces.

También podrán utilizarse otras cantidades:

```
section verse x3 {
    | C | Am | F | G |
}
```

La cantidad de repeticiones deberá ser un entero positivo.

Esto permite representar estructuras musicales repetitivas sin tener que duplicar manualmente los compases.

4.7. Compases

Los compases serán una de las construcciones principales del lenguaje.

Cada compás estará delimitado por el símbolo `|`:

```
| C |
```

Una sección podrá contener múltiples compases:

```
section verse {
    | C | Am | F | G |
}
```

Cada línea representa una secuencia de compases.

El símbolo  tendrá un significado específico dentro del dominio musical y será utilizado por el parser para identificar los límites de cada compás.

4.8. Acordes

El lenguaje permitirá representar acordes utilizando una notación cercana a la utilizada por músicos:

```
C
Am
F
G
```

La representación de un acorde estará compuesta conceptualmente por:

- Raíz.
- Cualidad.
- Extensiones opcionales.
- Bajo opcional.

Entre las cualidades podrán encontrarse:

- Mayor.
- Menor.
- Dominante.
- Disminuido.
- Aumentado.

Por ejemplo:

```
C
Am
G7
Fmaj7
Bdim
```

El compilador deberá poder interpretar estas construcciones como estructuras armónicas.

4.9. Acordes con extensiones

Los acordes podrán incluir extensiones o cualidades más específicas:

```
| Cmaj7 | Am7 | Dm7 | G7 |
```

Esto permitirá representar progresiones armónicas más expresivas sin necesidad de describir individualmente las notas que forman cada acorde.

La interpretación concreta de las extensiones será responsabilidad del compilador durante la generación del MIDI.

4.10. Slash chords

El lenguaje permitirá especificar acordes con una nota de bajo diferente de su raíz utilizando `/`.

Por ejemplo:

```
| C/E |
```

representa un acorde de C mayor con E como nota de bajo.

Otro ejemplo:

```
| Dm/F |
```

representa un acorde de D menor con F en el bajo.

Conceptualmente, un slash chord estará formado por:

```
acorde / nota_de_bajo
```

Esta información podrá utilizarse durante la generación del MIDI para producir una línea de bajo acorde con la armonía especificada.

4.11. Acordes sin duración explícita

Cuando un compás contenga un único acorde y no se indique una duración explícita, se asumirá que dicho acorde ocupa todo el compás.

Por ejemplo:

```
time 4/4
```

```
| C |
```

equivale conceptualmente a:

```
| C(4) |
```

Esta regla permite mantener una sintaxis simple para los casos más comunes.

4.12. Duraciones de acordes

Un compás podrá contener más de un acorde especificando la duración de cada uno mediante una cantidad de tiempos:


```
| C(2) G(2) |
```

En un compás de 4/4:

```
C → 2 tiempos  
G → 2 tiempos
```

Por lo tanto:

```
2 + 2 = 4
```

y el compás es válido.

También podrán utilizarse otras combinaciones:

```
| C(1) F(1) G(2) |
```

La suma deberá coincidir exactamente con la cantidad de tiempos disponible en el compás.

4.13. Validación temporal de compases

Una de las funcionalidades principales de B# será la validación de la duración de los compases.

Por ejemplo:

```
time 4/4  
| C(2) G(2) |
```

será aceptado porque la duración total es de cuatro tiempos.

En cambio:

```
time 4/4  
| C(3) G(2) |
```

será rechazado porque la duración total es de cinco tiempos.

También deberá rechazarse un compás cuya duración sea insuficiente:

```
time 4/4  
| C(2) G(1) |
```

porque:

```
2 + 1 = 3
```

en lugar de cuatro tiempos.

Esta validación será realizada durante el análisis semántico.

4.14. No Chord

El lenguaje permitirá indicar que no existe armonía durante un compás mediante:

```
| N.C. |
```

N.C. significa **No Chord**.

Esto permite representar secciones de una canción en las que no se especifica ningún acorde.

Por ejemplo:

```
section intro {  
  | N.C. | C | Am | G |  
}
```

Cuando **N.C.** aparezca como único contenido de un compás, se considerará que ocupa la totalidad de ese compás.

4.15. Reutilización de estructuras

La reutilización en B# estará centrada principalmente en las secciones musicales mediante su cantidad de repeticiones.

Por ejemplo:

```
section chorus x4 {  
  | F | G | C | C |  
}
```

permite representar un estribillo que será ejecutado cuatro veces.

De esta manera, el usuario no necesita escribir cuatro veces los mismos compases.

4.16. Generación del artefacto musical

El compilador recibirá como entrada un programa escrito en B#.

Como resultado de la compilación se generará una representación musical, principalmente un archivo **MIDI**, que contendrá la información armónica descrita por el usuario.

Para la generación del MIDI, los acordes podrán traducirse a las notas que los componen.

Por ejemplo:

```
| C |
```

podrá representarse mediante las notas correspondientes al acorde de C mayor.

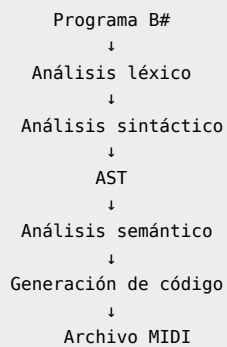
En el caso de un slash chord:

```
| C/E |
```

el MIDI podrá representar las notas del acorde de C mayor junto con E como nota de bajo.

El mecanismo concreto de generación del archivo MIDI será definido durante las etapas posteriores del proyecto.

Flujo esperado



5. Sistema de tipos

B# utilizará un sistema de tipos que combine tipos básicos con tipos específicos del dominio musical.

5.1. Tipos básicos

Integer

Representa números enteros.

Se utilizará, entre otros casos, para:

- Tempo.
- Cantidad de repeticiones.
- Duraciones expresadas en tiempos.

Ejemplo:

```
tempo 120
```

String

Representa cadenas de caracteres.

Se utilizará, por ejemplo, para el nombre de una canción:

```
song "Midnight" {  
    ...  
}
```

5.2. Tipos específicos del dominio

Chord

Representa un acorde musical.

Ejemplos:

```
C  
Am  
G7  
Cmaj7
```

Conceptualmente, un acorde podrá contener:

- Root.
- Quality.
- Extensions.
- Optional bass.

Duration

Representa la duración temporal de un acorde dentro de un compás.

Por ejemplo:

```
C(2)
```

indica que el acorde **C** tiene una duración de dos tiempos.

Bar

Representa un compás musical.

Un **Bar** estará compuesto por uno o más acordes con sus respectivas duraciones, o por **N.C.**.

Por ejemplo:

```
| C(2) G(2) |
```

Section

Representa una sección de una canción.

Una sección contiene una secuencia de compases y una cantidad de repeticiones.

Ejemplo:

```
section verse x2 {  
    | C | Am | F | G |  
}
```

Song

Representa la composición completa.

Contiene información como:

- Nombre.
- Tempo.
- Compás.
- Tonalidad.
- Secciones.

5.3. Tipado y validación semántica

Se propone utilizar tipado estático y realizar las validaciones durante la compilación.

Además de las comprobaciones de tipos, B# deberá realizar validaciones específicas del dominio.

Por ejemplo, deberá detectarse un acorde inválido:

```
| H |
```

y una duración incompatible con el compás:

```
time 4/4  
| C(3) G(2) |
```

También deberán validarse las cantidades de repeticiones:

```
section chorus x0 {  
    | C | G |  
}
```

La última construcción deberá ser rechazada porque una sección debe repetirse una cantidad positiva de veces.

6. Casos de prueba

Los siguientes casos se plantean como **Unit Tests**, buscando que cada uno sea compacto y específico para comprobar una funcionalidad determinada.

6.1. Casos de aceptación

A1 - Canción mínima

```
song "Test" {  
}
```

Resultado esperado: aceptación.

Verifica que pueda declararse una canción sin configuración adicional.

A2 - Configuración de tempo

```
song "Test" {  
    tempo 120  
}
```

Resultado esperado: aceptación.

Verifica la declaración de un tempo válido.

A3 - Configuración de compás

```
song "Test" {  
    time 4/4  
}
```

Resultado esperado: aceptación.

Verifica la definición de un compás válido.

A4 - Definición de tonalidad

```
song "Test" {  
    key C major  
}
```

Resultado esperado: aceptación.

Verifica la declaración de una tonalidad válida.

A5 - Sección con un acorde

```
song "Test" {  
    section verse {  
        | C |  
    }  
}
```

Resultado esperado: aceptación.

Verifica la definición de una sección con un compás válido.

A6 - Progresión armónica

```
song "Test" {  
  time 4/4  
  
  section verse {  
    | C | Am | F | G |  
  }  
}
```

Resultado esperado: aceptación.

Verifica una progresión armónica de varios compases.

A7 - Repetición de sección

```
song "Test" {  
  section chorus x2 {  
    | C | G | Am | F |  
  }  
}
```

Resultado esperado: aceptación.

Verifica la definición de una sección con repeticiones.

A8 - Dos acordes en un compás

```
song "Test" {  
  time 4/4  
  
  section verse {  
    | C(2) G(2) |  
  }  
}
```

Resultado esperado: aceptación.

Verifica que un compás pueda contener múltiples acordes y que sus duraciones sumen correctamente los cuatro tiempos.

A9 - Slash chord

```
song "Test" {  
  section verse {  
    | C/E |  
  }  
}
```

Resultado esperado: aceptación.

Verifica la representación de un acorde con una nota de bajo diferente de su raíz.

A10 - No Chord

```

song "Test" {
  section intro {
    | N.C. |
  }
}

```

Resultado esperado: aceptación.

Verifica la representación de un compás sin acorde.

A11 - Acordes con extensiones

```

song "Test" {
  section verse {
    | Cmaj7 | Am7 | Dm7 | G7 |
  }
}

```

Resultado esperado: aceptación.

Verifica la representación de acordes con extensiones.

A12 - Canción completa

```

song "Midnight" {
  tempo 120
  time 4/4
  key C major

  section verse x2 {
    | C | Am | F | G |
  }

  section chorus x2 {
    | F | G | Em | Am |
    | F | G | C | C |
  }
}

```

Resultado esperado: aceptación.

Verifica la combinación de las principales construcciones del lenguaje.

6.2. Casos de rechazo

R1 - Compás con duración excedente

```

song "Test" {
  time 4/4

  section verse {
    | C(3) G(2) |
  }
}

```


Resultado esperado: rechazo.

La duración total del compás es cinco tiempos, mientras que el compás dispone de cuatro.

```
3 + 2 = 5 ≠ 4
```

R2 - Compás con duración insuficiente

```
song "Test" {  
  time 4/4  
  
  section verse {  
    | C(2) G(1) |  
  }  
}
```

Resultado esperado: rechazo.

La duración total es de tres tiempos y no completa el compás de 4/4.

```
2 + 1 = 3 ≠ 4
```

R3 - Acorde inválido

```
song "Test" {  
  section verse {  
    | H |  
  }  
}
```

Resultado esperado: rechazo.

H no corresponde a una raíz musical válida dentro de la notación soportada.

R4 - Tempo inválido

```
song "Test" {  
  tempo -120  
}
```

Resultado esperado: rechazo.

El tempo debe ser un número entero positivo.

R5 - Repetición inválida

```
song "Test" {  
  section chorus x0 {  
    | C | G |  
  }  
}
```

```
}  
}
```

Resultado esperado: rechazo.

La cantidad de repeticiones debe ser mayor que cero.

R6 - Compás inválido

```
song "Test" {  
  time 4/3  
}
```

Resultado esperado: rechazo.

El denominador del compás debe corresponder a una potencia de dos.

R7 - Tonalidad inválida

```
song "Test" {  
  key H major  
}
```

Resultado esperado: rechazo.

La raíz **H** no corresponde a una nota válida.

R8 - Slash chord inválido

```
song "Test" {  
  section verse {  
    | C/H |  
  }  
}
```

Resultado esperado: rechazo.

La nota utilizada como bajo debe ser una nota musical válida.

7. Ejemplos

7.1. Ejemplo 1 - Canción simple

El siguiente programa representa una canción utilizando una estructura de lead sheet:

```
song "Midnight" {  
  tempo 120  
  time 4/4  
  key C major  
  
  section verse x2 {  
    | C | Am | F | G |  
  }  
}
```

```

    section chorus x2 {
      | F | G | Em | Am |
      | F | G | C | C |
    }
  }
}

```

¿Qué representa?

El programa define una canción llamada **Midnight**.

Se establece:

- Un tempo de 120 BPM.
- Un compás de 4/4.
- Una tonalidad de C mayor.

Luego se definen dos secciones:

- `verse`
- `chorus`

El verso contiene cuatro compases:

```
| C | Am | F | G |
```

y se repite dos veces.

El estribillo contiene ocho compases:

```
| F | G | Em | Am |
| F | G | C | C |
```

y también se repite dos veces.

La información puede ser transformada posteriormente en un archivo MIDI.

7.2. Ejemplo 2 - Múltiples acordes y slash chords

El siguiente ejemplo muestra algunas de las construcciones específicas del lenguaje:

```

song "Summer Lights" {
  tempo 128
  time 4/4
  key C major

  section verse {
    | C(2) G(2) |
    | Am(2) F(2) |
    | C/E |
    | G7 |
  }

  section chorus x2 {
    | Fmaj7 | G7 | Em7 | Am7 |
    | N.C. | G7 | C | C |
  }
}

```

```
}  
}
```

En este ejemplo:

```
| C(2) G(2) |
```

contiene dos acordes dentro del mismo compás.

Como el compás es 4/4:

```
2 + 2 = 4
```

por lo que el compás es válido.

Por otro lado:

```
| C/E |
```

representa un acorde de C mayor con E como nota de bajo.

Finalmente:

```
| N.C. |
```

indica que durante ese compás no se especifica ningún acorde.

Este ejemplo muestra cómo B# permite representar una estructura armónica de manera compacta y cercana a una notación musical utilizada por músicos.

8. Resumen de la propuesta

B# será un **DSL orientado a la descripción armónica y estructural de canciones mediante una representación similar a un lead sheet**.

Las principales abstracciones del lenguaje serán:

- Canciones.
- Secciones.
- Compases.
- Acordes.
- Duraciones.
- Tonalidades.
- Tempo.
- Compás.
- Repeticiones.

- Slash chords.
- Extensiones de acordes.
- **N.C.** para compases sin acorde.

Una de las principales características del lenguaje será la utilización del símbolo **|** para delimitar compases:

```
| C | Am | F | G |
```

Esto permite que la sintaxis sea visualmente cercana a una representación musical real.

Además, B# incorporará una validación semántica específica del dominio: **la duración de los acordes dentro de cada compás deberá coincidir con la cantidad de tiempos definida por el compás.**

Por ejemplo:

```
time 4/4
| C(2) G(2) |
```

será válido, mientras que:

```
| C(3) G(2) |
```

será rechazado.

El compilador seguirá el siguiente flujo:

```
Programa B#
↓
Análisis léxico
↓
Análisis sintáctico
↓
AST
↓
Análisis semántico
↓
Generación de código
↓
Archivo MIDI
```

El artefacto principal generado será un archivo **MIDI** construido a partir de la información armónica y estructural especificada en el programa.

La sintaxis presentada en este documento constituye la propuesta inicial de B# para la Stage I. Durante las etapas posteriores podrán realizarse ajustes de sintaxis que no modifiquen significativamente los conceptos fundamentales del lenguaje.